

Proposed fix — test 57 deterministic hardening (V2, anti-bluff, no product-defect masking)

Date: 2026-06-13 **Verdict (from rootcause.md):** V2 — two test-isolation defects. No product defect. The prefix `P` paste binding, `paste-buffer -p`, and server lifecycle are all sound (70/70 isolated part-D runs clean).

These fixes make test 57 **deterministic** without weakening any assertion — each one removes a non-product source of flake while keeping the real end-user proof intact. Apply all three; they are independent and small.

Target file: `scripts/tests/57_reload_select_copy_paste.sh` (and the identical `mktemp` + clipboard issues in `scripts/tests/56_real_mouse_drag_copy.sh`).

FIX 1 (REQUIRED — primary flake) — make part D not depend on the GLOBAL clipboard timing

Root cause: part D writes the macOS pasteboard (`pbcopy`, line 184) and reads it back via prefix `P` → `pbpaste` ~4 s later (line 193). Any concurrent `pbcopy` in that window overwrites the value (proven: pasted `NOISE_<n> / OSPASTE_<i>_...` foreign tokens). This is shared global mutable state.

The cleanest anti-bluff fix is **DON'T depend on `pbpaste` for the exact-value proof** — drive the paste through a deterministic, test-private `READ` source. The shipped `@clip-read` user option is exactly the OS-adaptive read hook the binding is meant to use, but the prefix `P` binding hard-codes `pbpaste` instead of consulting `@clip-read`. (That hard-coding is arguably a small source-side gap, but fixing the binding is out of scope here; the test can still be made deterministic at the test layer.)

Recommended (smallest deterministic change): re-verify-on-mismatch with a clipboard re-assert + single retry

Keep the real prefix `P` path, but make the clipboard write→read atomic from the test's view by (a) setting the clipboard immediately before the keypress with a short settle, and (b) on a mismatch, **re-assert the clipboard and retry the keypress ONCE** — then `FAIL` only if it still mismatches **AND** the value pasted is not the test's own token. This survives a single concurrent overwrite without hiding a real paste break.

Exact edit — replace the part-D python invocation + assertion block (current lines ~182–219) with a retry wrapper. Concretely, wrap the existing pbcopy → python prefix-P → capture sequence in a 2-attempt loop, re-running printf '%s' "\$DTOK" | pbcopy at the top of each attempt and starting a FRESH session per attempt:

```
if [ "$(uname -s)" = "Darwin" ] && command -v pbcopy >/dev/null
    2>&1; then
    DTOK="OSPASTE_$$_$(date +%s)"
    d_ok=0
    for _attempt in 1 2; do
        "$BIN" -L "$L" kill-server 2>/dev/null || true
        "$BIN" -L "$L" -f "$CONF" new-session -d -s s -x 80 -y 24
        sleep 0.4
        printf '%s' "$DTOK" | pbcopy
        sleep 0.3
        python3 - "$BIN" "$L" <<'PY'
        # ... unchanged PTY-attach + \x02 P driver ...
PY
        sleep 0.4
        PANE_JOINED="$("$BIN" -L "$L" capture-pane -p -t s 2>/dev/
        null | tr -d '\n')"
        if printf '%s' "$PANE_JOINED" | grep -q "$DTOK"; then
            d_ok=1; break; fi
        done
        if [ "$d_ok" = 1 ]; then
            echo "EVIDENCE (D): real 'prefix P' pasted the EXACT OS-
            clipboard value '$DTOK' into the pane"
        else
            echo "FAIL: 57(D) – prefix-P did not paste the exact
            clipboard value (got: $(printf '%s' "$PANE_JOINED" | head
            -c 120))"; fail=1
        fi
        "$BIN" -L "$L" kill-server 2>/dev/null || true
    else
        echo "SKIP-layer: 57(D) OS-clipboard prefix-P chain ..." #
        unchanged
    fi
```

Rationale: a single concurrent overwrite during one ~4 s window is unlikely to recur on the immediate retry with a fresh pbcopy; a GENUINE paste break fails both attempts → still FAILs. This does not mask a product defect — it only tolerates the one non-product race (foreign clipboard writer).

Alternative (stronger isolation, still real prefix P): private @clip-read source

If the team prefers full isolation over a retry, point the running server's read hook at a **test-private file** and have part D's binding read THAT instead of the global pasteboard, then drive prefix P and assert. This requires the binding to consult @clip-read (a one-line source change to scripts/tmux.conf.template line 191: read via #{@clip-read} instead of the hard-coded pbpaste/wl-paste/... chain).

That is a real product improvement (the binding already documents `@clip-read` as "the OS-adaptive READ counterpart") but is a larger change than test-only hardening; flag for a separate work item if pursued.

The task asked for the smallest test-only fix → use the retry wrapper above.

FIX 2 (REQUIRED — the mkstemp failed abort + (unclassified) + the

"server exited" full-suite surface) — correct the BSD-mkstemp template

Root cause: `/tmp/mt_app.XXXXXX.py` puts `.py` AFTER the `XXXXXX`, so BSD `mkstemp` (macOS) does NOT expand the placeholder and creates the LITERAL, shared, fixed filename — colliding across tests 56+57 and across reruns, and self-perpetuating when `set -eu` aborts before the trap can clean it.

Exact edit (line 50 in BOTH 57 and 56 — line 49 in 56):

```
# BEFORE (broken on BSD mkstemp — X's not at end, literal
      filename):
APP=$(mkstemp /tmp/mt_app.XXXXXX.py)

# AFTER (X's at end → real random name, portable BSD+GNU):
APP=$(mkstemp "${TMPDIR:-/tmp}/mt_app.py.XXXXXX")
```

Notes:

- Moving `.py` BEFORE the Xs makes the name unique per run (proven: `mkstemp /tmp/mt_app_ok.py.XXXXXX → /tmp/mt_app_ok.py.R06Vs6`). The python app is invoked by path, so a `.py.<rand>` suffix is fine (it never relies on the `.py` extension).
- Honour `$TMPDIR` so per-process temp dirs further reduce collision surface.
- This removes BOTH the abort AND the cross-test contamination, which removes the rare `server exited unexpectedly` full-suite surface (the message only appeared on the aborted/contaminated runs).

Optionally, also make the EXIT trap tolerate the abort path (defensive, not required once the template is fixed): the trap already `rm -f "$SINK" "$APP"`; with a valid unique `$APP` there is no shared file to delete out from under a sibling.

FIX 3 (RECOMMENDED — eliminate the orphan helper client that emits the

`benign server exited unexpectedly`) — reap the PTY-attach child

Root cause of the message itself: `inject_drag` (lines 77–108) and part D (186–203) fork a PTY-attach tmux client and `os.close(fd)` WITHOUT `os.waitpid(pid, 0)`. The orphan stays attached; when a later `kill-server` (line 171 / 220) tears the server down, the orphan prints `CLIENT_EXIT_LOST_SERVER = "server exited unexpectedly"` to its console.

Exact edit — in BOTH python blocks, after `os.close(fd)` add an explicit `detach+reap` so no client is attached when the server is later killed:

```
try: os.close(fd)
except OSError: pass
# §11.4.6: deterministically reap the attach client so it is gone
#         BEFORE any
# later kill-server, eliminating the benign LOST_SERVER ("server
#         exited
# unexpectedly") console notice from an orphaned client.
try:
    os.kill(pid, 15)
    # SIGTERM the attach client (detaches cleanly)
except (ProcessLookupError, OSError):
    pass
try:
    os.waitpid(pid, 0)
except (ChildProcessError, OSError):
    pass
```

This is hygiene only (the message is benign and does not reach `run_all`'s captured stdout — proven 0/30), but it removes the confusing console line that prompted this investigation and prevents accumulating orphan clients across a full-suite run.

Verification plan (anti-bluff, §11.4.50 determinism)

After applying FIX 1+2 (+3), confirm:

1. `rm -f /tmp/mt_app.* ; for i in $(seq 1 20); do bash scripts/tests/57_*.sh; done` under a concurrent `while true; do printf NOISE_$RANDOM | pbcopy; sleep .1; done` → **20/20 PASS** (FIX 1 retry survives the clipboard race; FIX 2 prevents abort). Pre-fix control (captured this session): part-D FAILED 5/15 under that noise.
2. Seed a stale `/tmp/mt_app.XXXXXX.py`, run 57 → must NOT abort (FIX 2).
3. `run_all.sh` full suite (with `TMUX_BIN` pointed at the darwin build so 57 actually executes) → 57 prints a real PASS/FAIL, never (unclassified).
4. No `server exited unexpectedly` line and no orphan tmux ... attach processes after the run (FIX 3): `pgrep -af 'tmux .*attach'` empty.

Do NOT do (would mask a product defect)

- Do NOT downgrade part D to a non-prefix P path or delete the exact-value assertion — the real-keypress OS-clipboard proof is the whole point of part D.
- Do NOT convert part D to an unconditional SKIP — part C proves the paste-buffer mechanism and part E proves the binding shape, but only part D proves

the END-TO-END real-prefix `P` → exact-OS-clipboard-value chain on macOS. Keep it; just make it race-robust (FIX 1).

- Do NOT silence `set -eu` around `mktemp` — fix the template (FIX 2), don't hide the abort.